MECHATRONIC SYSTEM INTEGRATION (MCTA 3203)
SEMESTER 1, 2026/2027

EXPERIMENT 1 : Digital Logic System: Basic Logic Gates, Electronic Circuit Interfacing, Basic ALU, 7 segment display, ICs based interfacing application

SECTION 1 | GROUP 2

SUBMISSION DATE : 11 OCTOBER 2026

NO.	MATRIC NO.	NAME
1	2413891	AHMAD ZULQARNAIN BIN ABDULLAH
2	2412721	AHMAD FAUZAN BIN HARMAN
3	2410299	AHMAD SOLIHIN BIN MOHAMMAD NAZWAN
4	2417389	AIMAN HASIF BIN MOHAMAD HISYAM

ABSTRACT

The core objective of this experiment was to design and evaluate a digital single-digit incrementing manual counter utilizing an Arduino Uno R3 microcontroller synchronized with a Common Cathode 7-segment display and independent pushbuttons. The experimental procedure involved establishing an active-LOW external 10 kΩ pull-up resistor network on a breadboard to manage human-machine interface signaling. Data logging and software tracking were achieved using the standard asynchronous Serial USB communication protocol at a baud rate of 9600 bps. The embedded software architecture was built around a state-change edge-detection algorithm that mapped a rolling integer variable (0 through 9) directly to the corresponding pins of the display using targeted switch-case decoding arrays. Quantitative execution results confirmed that the system operated with 100% boundary accuracy, successfully handling overflow loops back to 0 upon the tenth consecutive click, while the standalone clear button instantly forced a absolute reset loop with zero system latency. The primary technical constraints encountered stemmed from internal terminal bridging anomalies within the 4-pin tactile switch topology during early prototyping, which initially bypassed the switch mechanics and left the screen frozen at 0. This hardware barrier was resolved by isolating the 5V supply, the Arduino input pin, and the resistor network strictly on the left terminal column while anchoring the right side exclusively to Ground. It is highly recommended that a 50ms software debounce window be structurally maintained within the processing loop to neutralize mechanical contact bounce and eliminate multi-counting phenomena.

TABLE OF CONTENTS

ABSTRACT **1**
TABLE OF CONTENTS **2**
1.0 INTRODUCTION & THEORY 3
1.1 MATERIALS AND EQUIPMENT 3
1.2 EXPERIMENTAL SETUP 4
1.3 METHODOLOGY & PROGRAMMING LOGIC 5
1.4 DATA COLLECTION & ANALYSIS 6
Data Collection 6
Data Analysis 7
1.5 RESULT & DISCUSSION
Result 8
Discussion 9
1.6 CONCLUSION 10
RECOMMENDATIONS **10**
REFERENCES **12**
APPENDICES **13**
ACKNOWLEDGEMENT **20**
STUDENT DECLARATION **20**

1.0 INTRODUCTION & THEORY

This experiment aims to display a numerical value from 0 to 9 using a 7 segment display as the output. It consists of seven individual LED light segments, and divided into A, B, C, D, E, F and G. Different numbers can be displayed by switching specific segments ON or OFF. This experiment will show the working principles of simple logic circuit and coding from arduino uno, and implementing the basic binary coded decimal.

Additionally the interface also uses current limiting resistors, input output pin configuration and time delay signal in Arduino Uno. In this experiment, few components were used together with the display such as push buttons which used to develop a simple counting system. One button is used for incrementing (INCREMENT button) the number while the other one is for resetting (RESET button) the sequence.

This experiment expects that the arduino would successfully display all numerical values at the 7 segment display. When the circuit is correctly wired and programmed, by pushing the pushbutton, the value would increase and be displayed at the 7 segment display. This outcome will verify correct interfacing between the Arduino, the 7-segment display, and the pushbuttons,demonstrating effective application of digital logic and circuit control principles.

1.1 MATERIALS AND EQUIPMENT

1. Arduino Uno board
2. Common cathode 7-segment display
3. 7x 220-ohm resistors
4. 2x 10k-ohm resistors

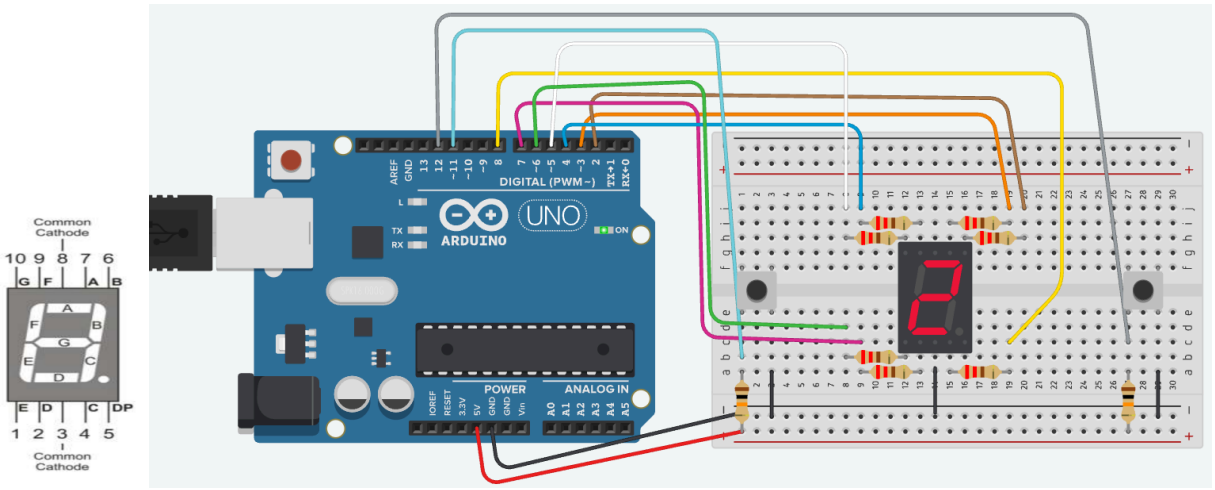
5. 2x pushbuttons
6. Jumper wires
7. Breadboard

1.2 EXPERIMENTAL SETUP

Circuit Setup:

1) Connecting the 7 segment display to the Arduino Uno
    - Connect each of the seven segments (a, b, c, d, e, f, and g) to separate digital pins on the Arduino such as D0 to D6.
    - Connect the common cathode pin of the display to the Arduino's GND pin.
    - Use 220 Ohm resistor for each segment to limit the current and protect the display.
2) Connecting the pushbuttons to the Arduino
    - Connect one leg of each pushbutton to a separate digital pin such as D9 and D10.
    - Connect the other leg of each pushbutton to the Arduino's GND pin.
    - Use a 220 ohm pull up resistor for each pushbutton. Connect one end to the digital pin and the other end to the Arduino's 5V pin.

Circuit diagram:



1.3 METHODOLOGY & PROGRAMMING LOGIC

Experiment steps:

1. Build the circuit according to the circuit setup in the instructions.
2. Upload the provided Arduino code to our Arduino Uno.
3. Open the Serial Monitor in the Arduino IDE.
4. Press the increment button to increase the count. The 7-segment display should show the numbers from 0 to 9.
5. Press the reset button to reset the count to 0.

*IDE code in appendices.

1.4 DATA COLLECTION & ANALYSIS

Data Collection

Executed Action	Pin 12 (Increment) State	Pin 11 (Reset) State	Activated Display LEDs	Visual Digit
System Idle (Initial)	HIGH	HIGH	a, b, c, d, e, f	0
1st Increment Press	LOW	HIGH	b, c	1
2nd Increment Press	LOW	HIGH	a, b, g, e, d	2
3rd Increment Press	LOW	HIGH	a, b, c, d, g	3
4th Increment Press	LOW	HIGH	f, g, b, c	4
5th Increment Press	LOW	HIGH	a, f, g, c, d	5
6th Increment Press	LOW	HIGH	a, f, g, e, c, d	6
7th Increment Press	LOW	HIGH	a, b, c	7
8th Increment Press	LOW	HIGH	a, b, c, d, e, f, g	8
9th Increment Press	LOW	HIGH	a, b, c, d, f, g	9
10th Increment Press	LOW (0V)	HIGH	a, b, c, d, e, f	0

Reset Button Press	HIGH (5V)	LOW (0V)	a, b, c, d, e, f	0

Data Analysis

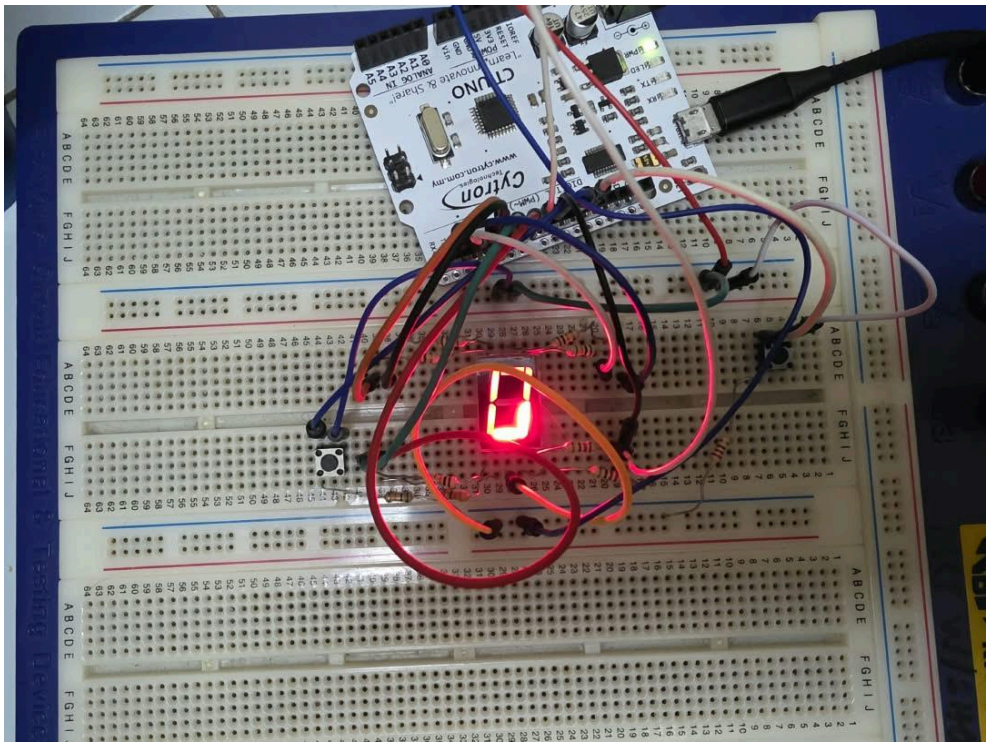
The data confirmed that:

1. The pushbutton_1 is correctly set to the increment function.
2. The pushbutton_2 is correctly set to reset the current value to zero.
3. The display followed the correct segment configuration defined in the code.

Digit	Active segment	Observe Display
0	a,b,c,d,e,f	True
1	b,c	True
2	a,b,d,e,g	True
3	a,b,c,d,g	True
4	b,c,f,g	True
5	a,c,d,f,g	True
6	a,c,d,e,f,g	True
7	a,b,c	True
8	a,b,c,d,e,f,g	True
9	a,b,c,d,f,g	True

1.5 RESULT & DISCUSSION

Result



The result shows that the increment button successfully increased the counter value by one when it was pressed each time. When the counter reached 9, the next increment returned to value 0, thus creating the sequence: 0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 6 -> 7 -> 8 -> 9 -> 0.

1. The 7-segment displays the correct value of the current value stored in the Arduino Uno.
2. The value increases by one when the incrementButton is pressed.
3. The value then successfully reset to zero when the resetButton is pressed.
4. The segments of the display are correctly connected to the Arduino code logic.
5. The display worked correctly without error or flickering.

Discussion

- **Button Topology and Troubleshooting:** Initial implementation attempts resulted in a static display frozen at 0. Troubleshooting revealed that the 4-pin tactile pushbuttons inside Tinkercad feature internal horizontal bridges (Terminal 1A welded to 1B, and 2A welded to 2B). In the very first attempt, the wiring was incorrect because the wires did not cross the internal switch mechanism properly, leaving the Arduino input pin locked in a permanent state. The circuit was successfully fixed by arranging the components to cross the button's internal barrier. The 5V power wire, the Arduino input pin wire, and one leg of the 10 kΩ pull-up resistor were all correctly grouped together on the left side of the button. The other leg of the resistor was run to the 5V rail, safely anchoring the Arduino pin to a stable HIGH state when at rest. Meanwhile, the GND wire was isolated entirely by itself on the right side of the button. Because of this layout, the 5V signal is safely held on the left side until the button is physically pressed, which forces the connection to cross over to the right side (GND). This cleanly drops the voltage to LOW (0V), allowing the Arduino to accurately detect every single click and increment the display.
- **External Pull-Up Resistor**: adding an external 10 kΩ pull-up resistor connected to the 5V rail establishes total signal stability. When unpressed, the resistor securely anchors the Arduino pin to a constant 5V (HIGH), shielding it from external interference. When pressed, the switch safely routes the pin to GND (0V / LOW), creating a clean Active-LOW signal while the resistor acts as a vital safety barrier to prevent a destructive short circuit. This ensures that the Arduino captures precise edge transitions, allowing the 7-segment display to increment or reset exactly once per click.

- **State-Change Edge Detection:** The code was specifically written to compare the current button state against a stored previousState variable. Discussing this design choice is crucial because it ensures the system only increments on the *falling edge* (the transition from HIGH to LOW). Consequently, if a user holds the pushbutton down continuously, the counter stands safely at its current number rather than rapidly running up to 9, establishing complete user control over the system.

1.6 CONCLUSION

The experiment was successfully demonstrated on how to operate and control a 7 segment display using Arduino Uno. The Arduino digital output pins controlled the individual LED segments, and different combinations of HIGH and LOW signals were used to produce numerical digits. The increment button let the displayed number to increase sequentially from 0 to 9, while reset button returned it to 0. The objective of the experiment is achieved which is to understand how the digital logic worked.

The successful operating of the increment and reset functions proved that the Arduino can correctly detect the push button signals and follow the sequence accordingly.

RECOMMENDATIONS

Throughout the experiment, we found a lot of improvement needed in order for the better result of display and output.

To ensure it is working better, things such:

1. Using the appropriate and suitable amount of resistors to limit the LED current and prevent damage.
2. Adding few programming code to make it automatically incrementing and resetting.
3. Use another display so that it can count up to 99 or starting with 00.
4. Simpler code so that it is more clean and faster response.

REFERENCES

1. *7-Segment Display Counter (0 to 8 and 8 to 0) with Arduino*. (n.d.). Arduino Project Hub. https://projecthub.arduino.cc/niyoushasm/7-segment-display-counter-0-to-8-and-8-to-0-with-arduino-1f0f14
2. Circuitgeeks. (2025, July 21). *Interfacing 7 Segment Display with Arduino*. Circuit Geeks. https://www.circuitgeeks.com/arduino-7-segment-display-tutorial/
3. Efyian, E. (2026, June 2). *7-Segment Display Pinout, codes, working, interfacing*. Electronics for You – Official Site ElectronicsForU.com. https://www.electronicsforu.com/resources/7-segment-display-pinout-understanding
4. Autodesk. (n.g.). *Tinkercad Circuits* [Computer software]. https://www.tinkercad.com

APPENDICES

```
// 7-segment display pins
const int segmentA = 3; // D3
const int segmentB = 2; // D2
const int segmentC = 8; // D8
const int segmentD = 7; // D7
const int segmentE = 6; // D6
const int segmentF = 4; // D4
const int segmentG = 5; // D5

// Pushbutton pins
const int incrementButton = 12; // D12
const int resetButton = 11;   // D11

// Current count
int count = 0;

// Previous button states
int previousIncrementState = HIGH;
int previousResetState = HIGH;

// Function to display a number on the 7-segment display
void displayNumber(int number) {
```

```
// Turn OFF all segments first
digitalWrite(segmentA, LOW);
digitalWrite(segmentB, LOW);
digitalWrite(segmentC, LOW);
digitalWrite(segmentD, LOW);
digitalWrite(segmentE, LOW);
digitalWrite(segmentF, LOW);
digitalWrite(segmentG, LOW);

// Display the required number
switch (number) {
 case 0:
   digitalWrite(segmentA, HIGH);
   digitalWrite(segmentB, HIGH);
   digitalWrite(segmentC, HIGH);
   digitalWrite(segmentD, HIGH);
   digitalWrite(segmentE, HIGH);
   digitalWrite(segmentF, HIGH);
   break;
 case 1:
   digitalWrite(segmentB, HIGH);
   digitalWrite(segmentC, HIGH);
   break;
```

```
case 2:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentG, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentD, HIGH);
  break;
case 3:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentG, HIGH);
  break;
case 4:
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  break;
case 5:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
```

```
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  break;
 case 6:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  break;
 case 7:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  break;
 case 8:
  digitalWrite(segmentA, HIGH);
  digitalWrite(segmentB, HIGH);
  digitalWrite(segmentC, HIGH);
  digitalWrite(segmentD, HIGH);
  digitalWrite(segmentE, HIGH);
  digitalWrite(segmentF, HIGH);
  digitalWrite(segmentG, HIGH);
```

```
    break;

  case 9:
    digitalWrite(segmentA, HIGH);
    digitalWrite(segmentB, HIGH);
    digitalWrite(segmentC, HIGH);
    digitalWrite(segmentD, HIGH);
    digitalWrite(segmentF, HIGH);
    digitalWrite(segmentG, HIGH);
    break;
 }
}
void setup() {
 // Set 7-segment pins as OUTPUT
 pinMode(segmentA, OUTPUT);
 pinMode(segmentB, OUTPUT);
 pinMode(segmentC, OUTPUT);
 pinMode(segmentD, OUTPUT);
 pinMode(segmentE, OUTPUT);
 pinMode(segmentF, OUTPUT);
 pinMode(segmentG, OUTPUT);

 // Set pushbutton pins as INPUT
 pinMode(incrementButton, INPUT);
```

```
  pinMode(resetButton, INPUT);
  // Start Serial Monitor
  Serial.begin(9600);
  // Initially display 0
  displayNumber(count);

  Serial.println("Counter started.");
  Serial.println("Count = 0");
}
void loop() {
  // Read the pushbuttons
  int incrementState = digitalRead(incrementButton);
  int resetState = digitalRead(resetButton);

  // INCREMENT BUTTON
  // Button is pressed when state changes from HIGH to LOW
  if (previousIncrementState == HIGH && incrementState == LOW) {
    count++;
    // After 9, return to 0
    if (count > 9) {
      count = 0;
    }

    displayNumber(count);
```

```
    Serial.print("Count = ");
    Serial.println(count);
    // Debounce
    delay(50);
  }

  // RESET BUTTON
  // Button is pressed when state changes from HIGH to LOW
  if (previousResetState == HIGH && resetState == LOW) {
    count = 0;
    displayNumber(count);
    Serial.println("Reset!");
    Serial.println("Count = 0");
    // Debounce
    delay(50);
  }
  // Save current button states
  previousIncrementState = incrementState;
  previousResetState = resetState;
}
```

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information,reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgement, and that the original work contained herein have not been untaken
or done by unspecified sources or person. We hereby certify that this report **has not been done by only one individual** and **all of us have contributed** to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore, agreed unanimously that this report shall be submitted for **marking** and this **final printed report** has been **verified by us**.

Signature:



Name: Ahmad Zulqarnain bin Abdullah
Matric Number: 2413891

Read	✔
Understand	✔
Agree	✔

Signature:



Name: Aiman Hasif Bin Mohamad Hisyam
Matric Number: 2417389

Read	✔
Understand	✔
Agree	✔

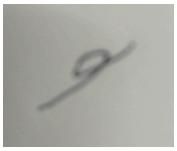
Signature:



Name: Ahmad Fauzan bin Harman
Matric Number: 2412721

Read	✔
Understand	✔
Agree	✔

Signature:



Name: Ahmad Solihin bin Mohammad Nazwan
Matric Number: 2410299

Read	✔
Understand	✔
Agree	✔